SLTC

(CCS3309) Big Data Assignment 02

Module leader : Mr. Chameera De Silva

Teaching Assistant : Mr. Chamod Hewage

Evaluation Panel
Student name: **Student ID:** **Student email:** **Group: 8** **Submission Date: 2026-February-19**
Module leader's feedback Dear Students, Good evening, and thank you for sharing your draft. I have reviewed your document briefly, and I'm pleased to say that you are on the right track overall. The structure is well aligned with the assignment requirements and covers the full pipeline clearly (Medallion architecture, PySpark processing, MongoDB modeling, indexing, analytics, and performance optimization). Regarding the length (36 pages): There is no issue with the page count as long as the content is meaningful and directly related to the tasks. From what I observed, most of the length comes from: Code screenshots MongoDB outputs Architecture and execution evidence This is acceptable for a technical Big Data submission.However, if you want to safely reduce the size (recommended target: ~25–30 pages), you may trim the following areas without losing marks: Reduce large code screenshots. Keep only key parts Move full code to Appendix (optional) Avoid repeating explanations of basic Spark/MongoDB commands Explain the purpose, not line-by-line syntax Limit multiple similar output screenshots Keep one representative result per task Make explanations concise Focus on business justification + technical reasoning Strengths observed: Clear Medallion architecture and business context Strong Feature Engineering (Revenue, Time features, RFM) Good MongoDB design (fact + dimensions) Proper indexing with justification Business insights are well written Overall assessment: Good quality work – continue in this direction. You do not need to worry about marks due to length, but trimming some repetitive technical evidence will improve readability and professionalism.

Contents

GDrive Link(Colab Notebook, Exported .ipynb file, Data quality Report & README) :
https://drive.google.com/drive/folders/1aehmCqV2BnTVrNv34U2XVSZ7Ye72-XNe?usp=sharing

Git repository link :
https://github.com/lakshanaat99/BigData_Final_Assignment/tree/4feee4e3b03cff06742d588d25d4a8458fe2d246

Introduction & business context

In the competitive landscape of modern e-commerce, organizations generate massive volumes of transactional data that hold critical insights into consumer behavior. For an international retailer, a manual or localized approach to data analysis is insufficient for high-velocity decision-making. The implementation of this automated PySpark-to-MongoDB pipeline addresses several core business needs such as personalized marketing, inventory & trend management, operational risk mitigation, revenue optimization.

To ensure data quality, scalability, and reliability, the system is structured using a multi-layered Medallion architecture.

Bronze Layer (Raw Data): Data is ingested from the raw CSV and stored in Parquet format.

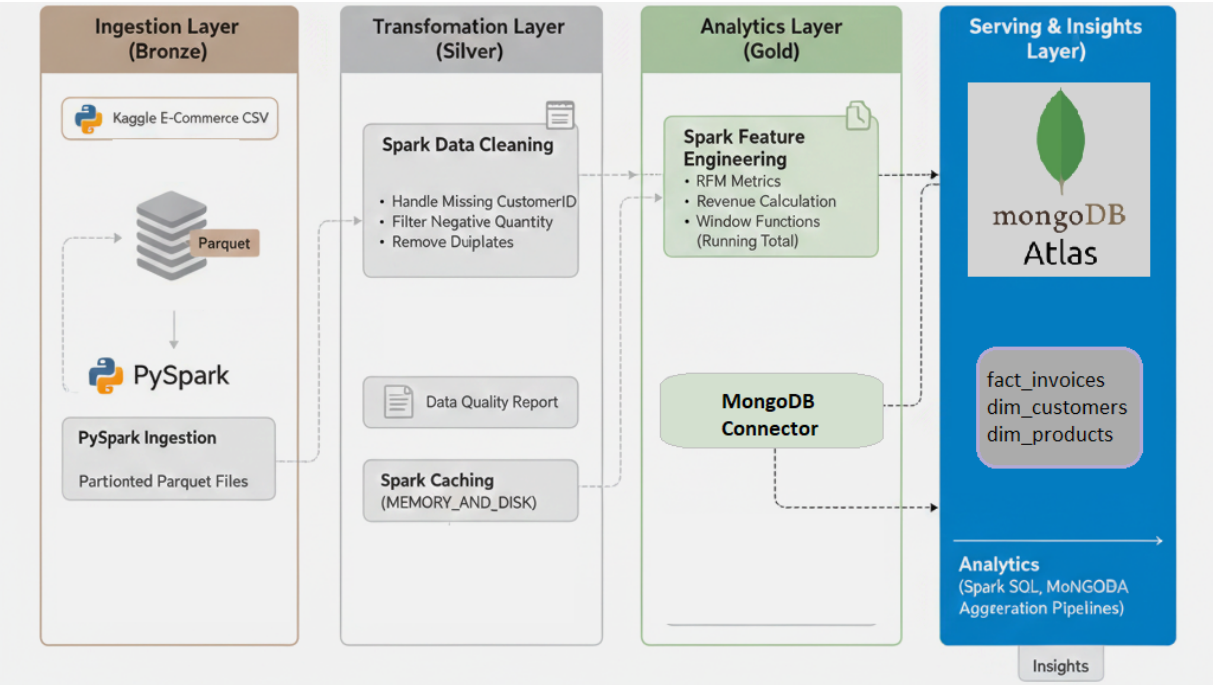
Silver Layer (Cleaned and Validated Data): This layer enforces schema integrity, handles missing values, and removes duplicates.

Gold Layer (Analytics-Ready Data): Feature-engineered datasets stored in MongoDB Atlas for rapid analytical querying.

Dataset Description

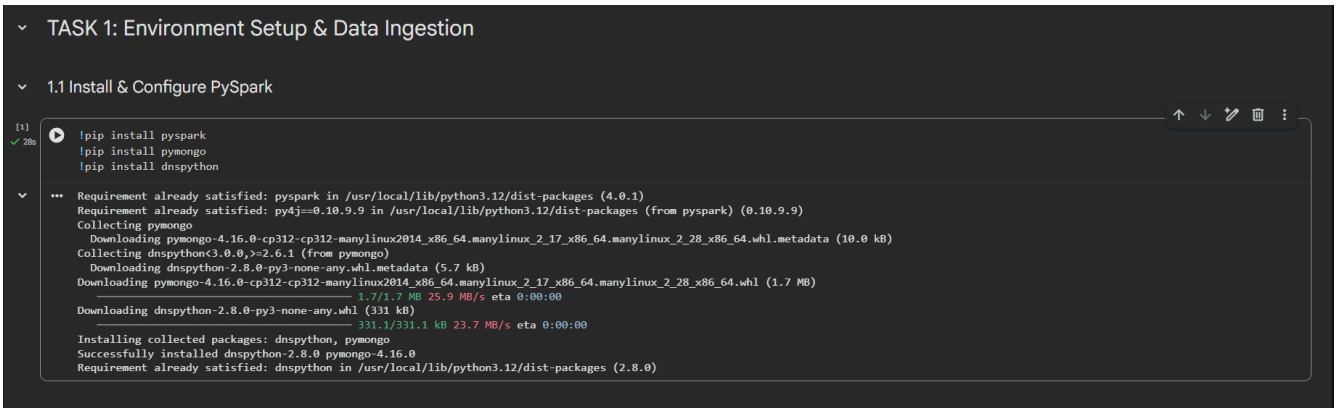
The dataset comprises transactional records from a retailer. It contains eight primary attributes, including Invoice, StockCode, Description, Quantity, InvoiceDate, Price, Customer ID, and Country. Many of the company's customers seem to be wholesalers, leading to large transaction volumes and varying quantities per invoice.

System architecture



Task 1 Environment Setup & Data Ingestion

1.1 Configure PySpark and MongoDB in Colab



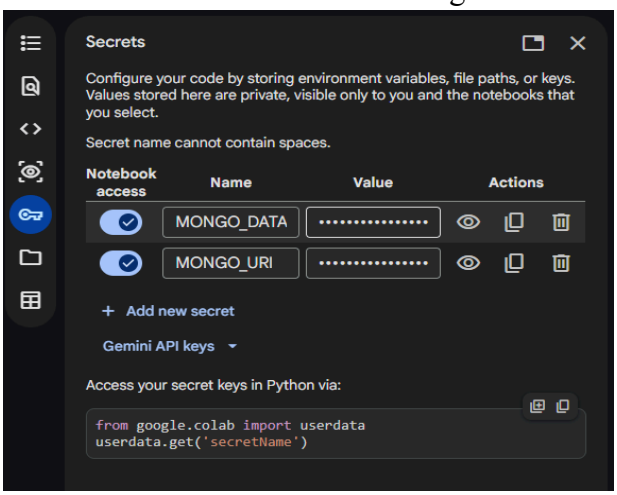
Install the necessary Python libraries using pip. The ! symbol indicates that the command is executed from the system (like a terminal) within a notebook. The first command installs the PySpark library version 3.5.1, which is used for handling big data. The second command installs the PyMongo library, which assists the Python programming language in connecting to a MongoDB database. The third command installs the dnspython library, which assists in handling internet connections required for MongoDB.

Creates a SparkSession, which is the starting point for using Spark in our program. First, it imports SparkSession from pyspark.sql. Then it builds a SparkSession and gives the application a name called "ECommerce-BigData-Pipeline". The .config() part adds the MongoDB Spark connector package, which allows Spark to connect to a MongoDB database. Finally, .getOrCreate() creates a new Spark session if one does not exist, or uses the existing one if it is already created.

Next our MongoDB configuration ensures a secure, production-grade connection by retrieving credentials from Google Colab userdata to prevent hard-coding sensitive information. These secrets, specifically the MONGO_URI and MONGO_DATABASE, are injected into the system's environment variables to allow the Spark-MongoDB Connector to authenticate automatically during the Gold layer write operations. Finally, a connectivity test is performed using the PyMongo MongoClient to execute an admin "ping" command , confirming that authentication is successful and the Atlas cluster is ready for data integration before any Spark transformations are executed.

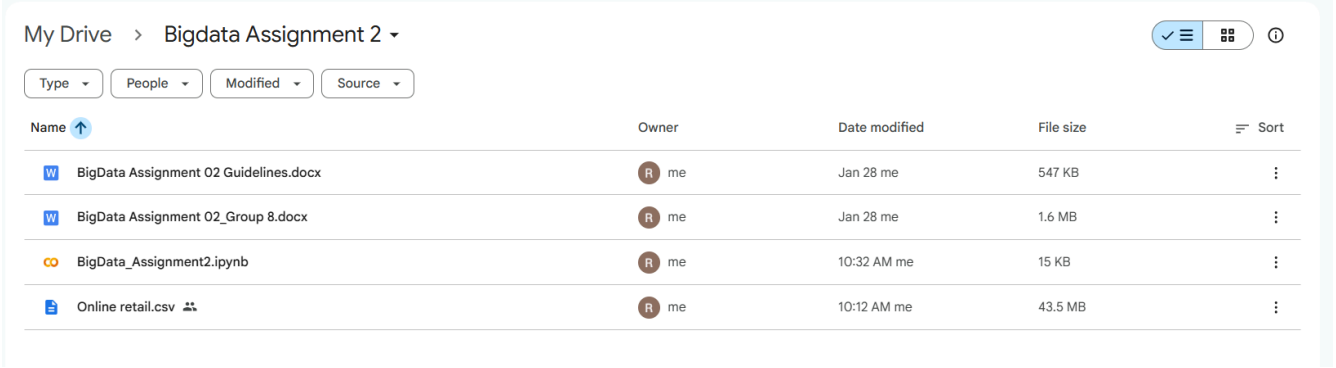
(For the complete PySpark implementation, please refer to Appendix A.1).

Secrets stored in secrets without hardcoding:



1.2 Load the dataset into Spark

Upload dataset to Google Drive → folder Bigdata Assignment 2



Mount the Drive and load the dataset

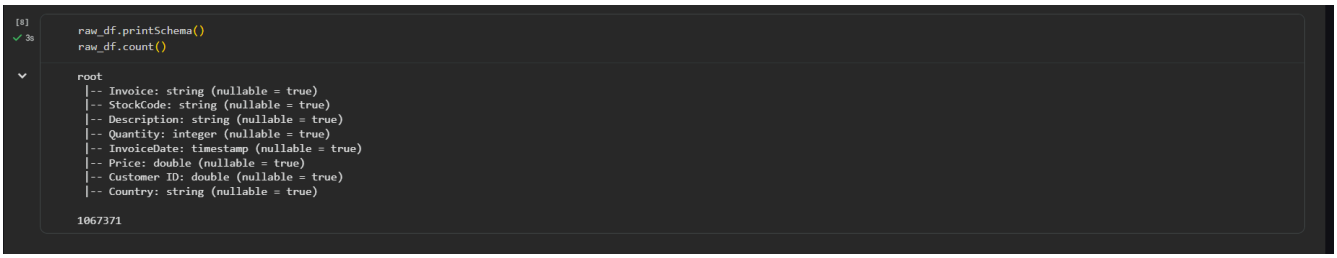
```
[3]
✓ 40s
# Connect Google Drive in Colab
from google.colab import drive
drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

[4]
✓ 11s
# Import SparkSession class from PySpark SQL module
from pyspark.sql import SparkSession

# Read the CSV file into a Spark DataFrame
raw_df = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .csv("/content/drive/MyDrive/Bigdata Assignment 2 /Online retail.csv")
```

First connect Google Drive to Google Colab. Imports the drive module from google.colab and then uses drive.mount('/content/drive') to link our Google Drive so we can access our files inside Colab.

After that, imports the SparkSession class again from PySpark. Then it reads a CSV file into a Spark DataFrame called raw_df. The .option("header", "true") tells Spark that the first row of

the CSV file contains column names. The .option("inferSchema", "true") tells Spark to automatically detect the data types of each column (like integer, string, etc.). Finally, the .csv() part gives the file path of the CSV stored in Google Drive. This means the data from "Online retail.csv" is now loaded into Spark for processing.

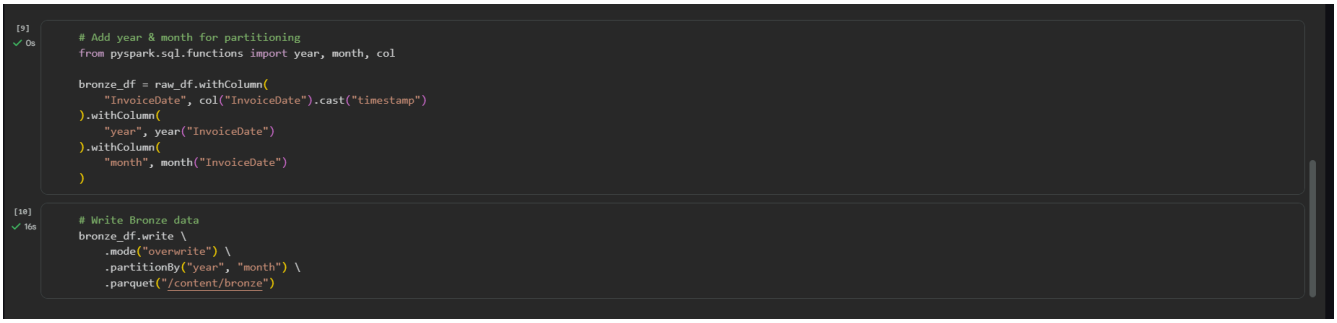
1.3 Display schema and record counts



The raw_df.printSchema() line shows the structure of our DataFrame. It displays the column names and their data types (like string, integer, double, etc.). This helps us understand how Spark has organized data.

The raw_df.count() line counts the total number of rows in the DataFrame. It tells how many records are in the dataset.

1.4 Store Bronze data as partitioned Parquet



First imports some useful Spark functions like year, month, col, and to_timestamp. These functions help modify and work with date columns.

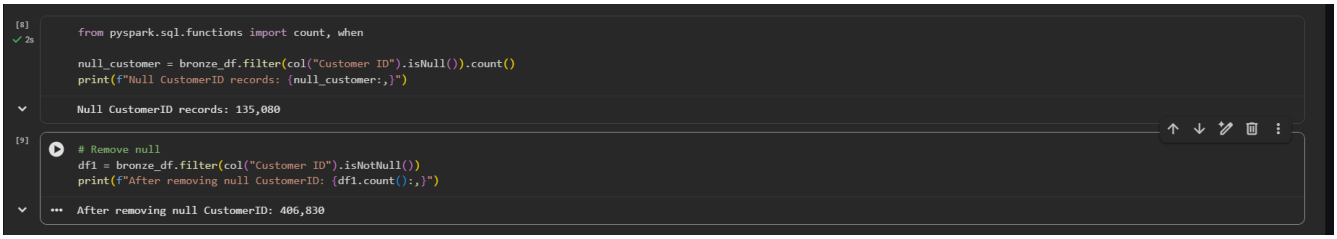
Then, a new DataFrame called bronze_df is created from raw_df. The withColumn() function is used to change the "InvoiceDate" column into a proper timestamp format using to_timestamp(). After that, two new columns called year and month are created from the "InvoiceDate" column. This makes it easier to organize the data by year and month.

Finally, the data is saved using .write(). The .mode("overwrite") means it will replace existing data if it already exists. The .partitionBy("year", "month") organizes the data into folders based on year and month. The .parquet("/content/bronze") saves the data in Parquet format inside the folder named "bronze".

Task 2 Data Cleaning & Quality Management

Implement and justify:

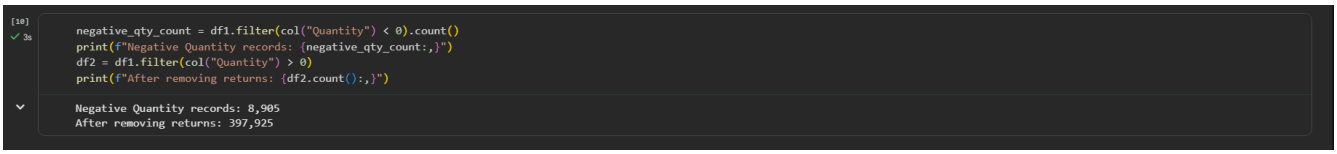
- Missing CustomerID handling



First, it imports the functions count and when from Spark. Then, it filters the bronze_df DataFrame to find rows where **Customer ID is null (empty)**. The .count() function counts how many records have a missing Customer ID, and it prints that number.

After that, it removes those null records using filter(col("Customer ID").isNotNull()). This keeps only the rows where Customer ID has a value. Finally, it counts and prints the number of remaining records after removing the null values.

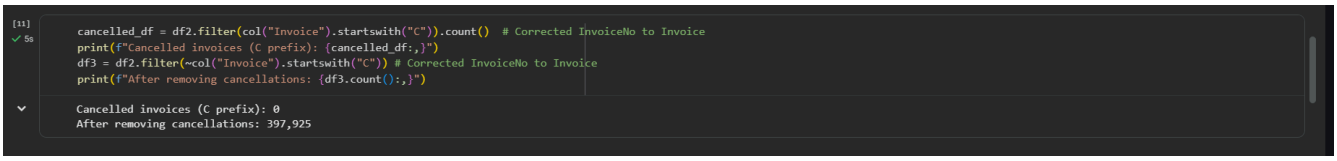
- Negative quantities (returns)



Filters the data to find rows where the quantity is less than 0. These usually represent returned items. Then counts how many such records exist and prints the number.

After that, create a new DataFrame called df2 that keeps only rows where the quantity is greater than 0. This removes returned or negative transactions. Finally, count and print how many records remain after removing those negative quantities.

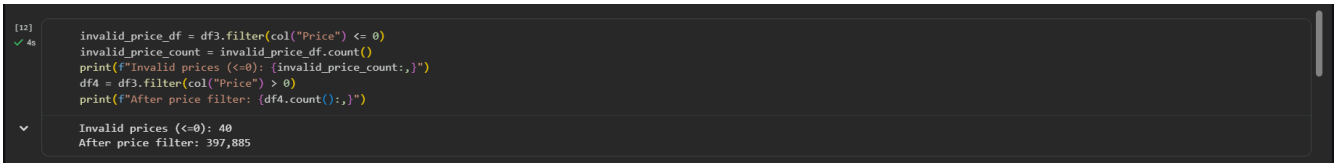
- Cancelled invoices



filters the DataFrame to find invoices where the **Invoice number starts with "C"**. In many retail datasets, invoices starting with "C" mean the order was cancelled. Counts how many cancelled invoices exist and prints that number.

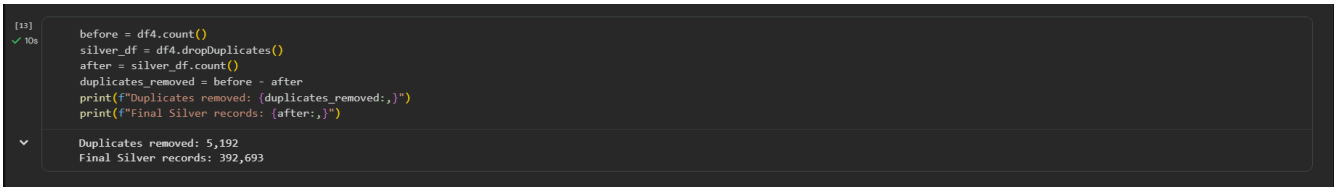
Then, create a new DataFrame called df3 that removes those cancelled invoices. The ~ symbol means “not”, so it keeps only invoices that do **not** start with "C". Finally, count and print how many records remain after removing the cancelled transactions.

- Invalid or extreme prices



Cleans the dataset by removing invalid prices. This first finds all rows where the **Price** is zero or negative, counts them, and shows how many records have invalid prices. Then create a new DataFrame that keeps only rows with positive prices, effectively removing all invalid entries. Finally, count and print how many records are left after this filtering step.

- Duplicate records



Removes duplicate rows from the dataset. First, it counts how many rows are in df4 before removing duplicates. Then it creates a new DataFrame called silver_df using .dropDuplicates(), which removes any repeated records. After that, it counts the rows again and calculates how many duplicates were removed. Finally, it prints the number of duplicates removed and the total number of records in the cleaned “Silver” dataset.

Data Quality Report

```
[14]
✓ 3s
dq_report_df = spark.createDataFrame(
    [{ "Metric": "Total Raw Records", "Count": raw_df.count() },
    { "Metric": "Null CustomerID Removed", "Count": null_customer },
    { "Metric": "Negative Quantity Removed", "Count": negative_qty_count },
    { "Metric": "Cancelled Invoices Removed", "Count": cancelled_df },
    { "Metric": "Invalid Prices Removed", "Count": invalid_price_count },
    { "Metric": "Duplicates Removed", "Count": duplicates_removed },
    { "Metric": "FINAL SILVER Records", "Count": after }],
    ["Metric", "Count"])

print("\n DATA QUALITY REPORT")
dq_report_df.show(truncate=False)
```

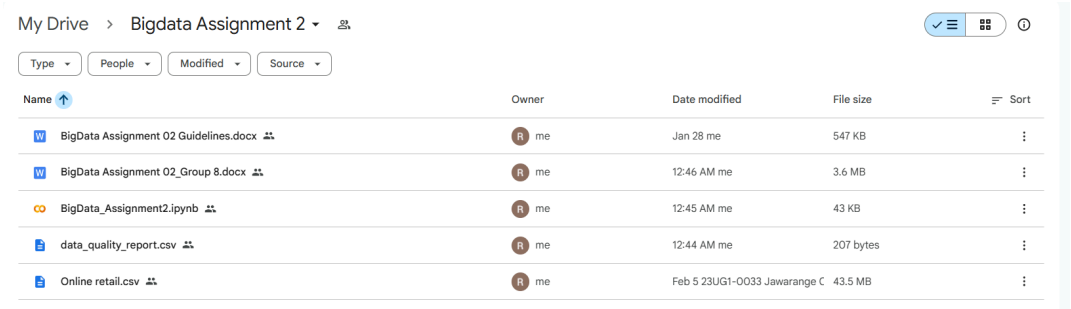
```
 DATA QUALITY REPORT
+------+-------------------------+
|Metric|Count                    |
+------+-------------------------+
|541910|Total Raw Records        |
|135080|Null CustomerID Removed  |
|8905  |Negative Quantity Removed|
|0     |Cancelled Invoices Removed|
|40    |Invalid Prices Removed   |
|5192  |Duplicates Removed       |
|392693|FINAL SILVER Records     |
+------+-------------------------+
```

Creates a **Data Quality (DQ) report** for the dataset. This makes a new Spark DataFrame called dq_report_df with two columns: **Metric** and **Count**. Each row shows a key step in cleaning the data, like how many null Customer IDs were removed, how many negative quantities, cancelled invoices, invalid prices, and duplicates were removed, along with the total raw records and the final number of cleaned "Silver" records.

Finally, print the title "DATA QUALITY REPORT" and use .show() to display the report in a clear table format without truncating the text. This helps quickly see the effect of all data cleaning steps.

```
[38]
✓ 0s
import os
import pandas as pd

# Path to save CSV
folder_path = '/content/drive/MyDrive/Bigdata Assignment 2'
output_path = os.path.join(folder_path, 'data_quality_report.csv')

# Create the folder if it does not exist
os.makedirs(folder_path, exist_ok=True)

# Export the Data Quality report
try:
    dq_report_df.toPandas().to_csv(output_path, index=False)
    print(f"\nData Quality Report saved as '{output_path}'")
except Exception as e:
    print(f"\nCould not save to Drive. Error: {e}")
```

```
Data Quality Report saved as '/content/drive/MyDrive/Bigdata Assignment 2/data_quality_report.csv'
```

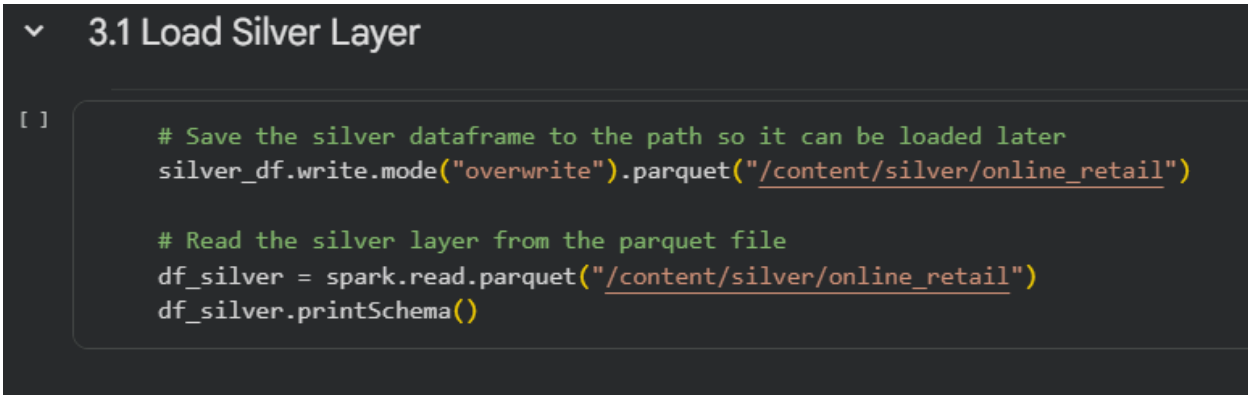


Export data quality report to drive as .csv

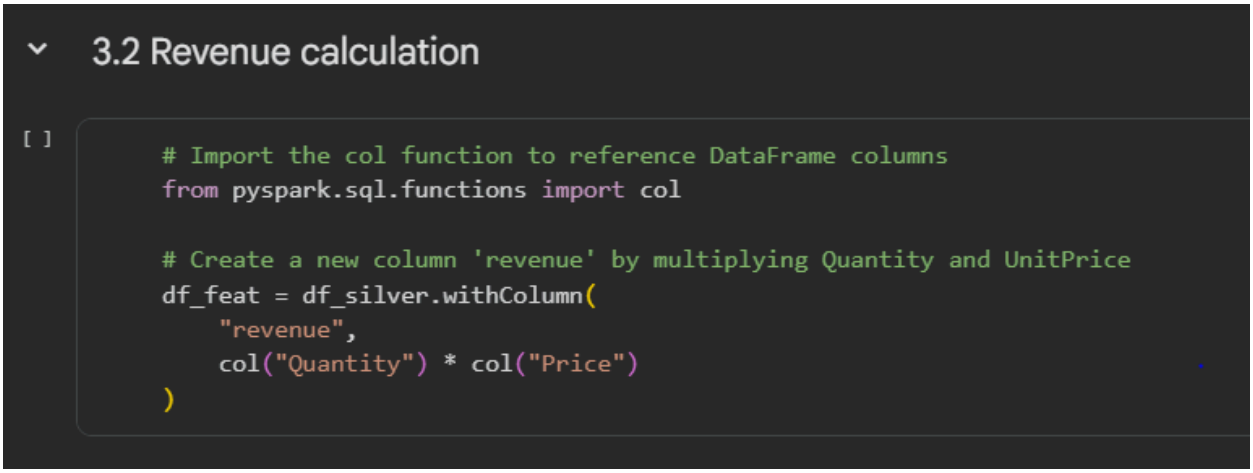
Task 3 Feature Engineering

Introduction for task 3

- The cleaned Silver layer dataset was feature engineered to convert transactional data into analytics ready data. This task was aimed at developing some meaningful business features that will allow analyzing customer behavior, tracking revenue and sales analysis by time.



The silver dataset, created after data cleaning and validation, was stored in parquet format and reloaded for feature engineering. This step ensures that feature engineering begins from a clean and validated dataset.



A new feature called revenue was created to measure the monetary value of every line of transactions.

- **Revenue = Quantity x Price**

Business Importance: Allows sales performance analysis, Full invoice and customer level aggregation, Develops the foundation for RFM metrics.

3.3 Time-based features (hour, weekday, month)

```
[ ]
# Import time-related functions from PySpark
from pyspark.sql.functions import hour, dayofweek, month

# Extract hour, day of week, and month from the InvoiceDate column
df_feat = df_feat.withColumn("invoice_hour", hour("InvoiceDate")) \
                 .withColumn("weekday", dayofweek("InvoiceDate")) \
                 .withColumn("invoice_month", month("InvoiceDate"))
```

Time related attributes were extracted from the InvoiceDate column to analyze purchasing behavior over time.

The following features were created:

- **invoice_hour** - Identifies peak shopping hours
- **weekday** - Analyzes weekday vs weekend trends
- **invoice_month** - Supports monthly revenue trend analysis

Business Importance: These features help management to understand the purchasing patterns of the customers over time.

3.4 Basket-level metrics

```
[ ]
# Import aggregation functions from PySpark
from pyspark.sql.functions import sum, countDistinct

# Group by Invoice to calculate total revenue and number of unique items per basket
basket_df = df_feat.groupBy("Invoice").agg(
    sum("revenue").alias("basket_revenue"),
    countDistinct("StockCode").alias("items_per_basket")
)

# Join the basket metrics back to the main features dataframe
df_feat = df_feat.join(basket_df, on="Invoice", how="left")
```

Basket-level features summarize purchasing behavior at the invoice level.

Two new features were created:

- **basket_revenue** - Total revenue per invoice
- **items_per_basket** - Number of unique products purchased per invoice

Business Importance: Identifies high value orders, Supports basket size analysis, Useful for cross-selling strategies.

3.5 Customer RFM features

```
# Import functions for calculating dates and maximum values
from pyspark.sql.functions import max, datediff, current_date, countDistinct, sum

# Group by Customer ID to calculate RFM (Recency, Frequency, Monetary) metrics:
# Recency: Days between today and the last purchase
# Frequency: Count of unique invoice numbers
# Monetary: Sum of total revenue
rfm_df = df_feat.groupBy("Customer ID").agg(
    datediff(current_date(), max("InvoiceDate")).alias("recency"),
    countDistinct("Invoice").alias("frequency"),
    sum("revenue").alias("monetary")
)

# Join the customer-level RFM features back to the main dataframe
df_feat = df_feat.join(rfm_df, on="Customer ID", how="left")
```

RFM (Recency, Frequency, Monetary) analysis was implemented to model customer purchasing behavior.

The RFM metrics represent:

- **Recency** - Days since last purchase
- **Frequency** - Number of unique invoices
- **Monetary** - Total revenue generated by the customer

Business Importance: RFM is widely used for customer segmentation, Identifying loyal customers, Predicting customer lifetime value.

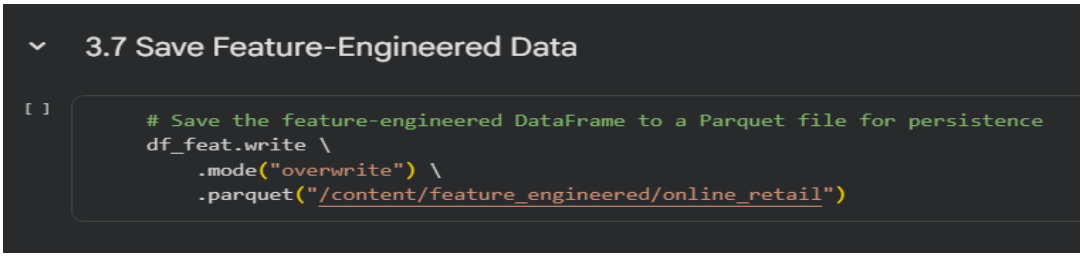
3.6 Window-Based Feature

```
# Import Window functionality and the sum aggregation function
from pyspark.sql.window import Window
from pyspark.sql.functions import sum

# Define a window partitioned by Customer ID and ordered by InvoiceDate
# It covers all rows from the start of the customer's history up to the current row
customer_window = Window.partitionBy("Customer ID") \
                        .orderBy("InvoiceDate") \
                        .rowsBetween(Window.unboundedPreceding, Window.currentRow)

# Calculate a running total of revenue for each customer using the defined window
df_feat = df_feat.withColumn(
    "running_customer_spend",
    sum("revenue").over(customer_window)
)
```

To analyze how customer spending evolves over time, a window function was applied.

- **running_customer_spend** - cumulative revenue per customer ordered by time

  Business Importance: Tracks spending growth, Identifies high-value repeat customers, Demonstrate use of distributed window processing in Spark.



The enriched dataset was saved as a Parquet file to prepare for Gold-level storage in MongoDB. This ensures persistence and enables reuse in subsequent tasks.

Conclusion of task 3

- Feature engineering transformed raw transactional records into analytics-ready data enriched with business intelligence indicators. **Revenue, time-based attributes, basket metrics, RFM analysis and window-based cumulative spending** collectively provide a strong foundation for customer analytics and performance evaluation.

  These engineered features were persisted in Parquet format to support Gold-layer storage in MongoDB.

Task 4 MongoDB Data Modeling

- The objective of task 4 is to design and implement three MongoDB collections using the feature engineered dataset created in the earlier task 3. (MongoDB was used as the Gold Layer storage)

  The required collections are:

  1. **fact_invoices** - Invoice-level data with embedded line items
  2. **dim_customers** - Customer-level summary with RFM metrics and segmentation
  3. **dim_products** - Product-level performance with country-wise distribution

  To connect Spark with MongoDB Atlas, a Spark session was created using the MongoDB connector.

  (For the complete PySpark implementation, please refer to Appendix A.2).

The dataset generated in Task 3 was loaded from the Parquet file. This dataset contains revenue, time features, RFM metrics and window-based features.

4.3 COLLECTION 1 - fact_invoices

Create invoice-level structure

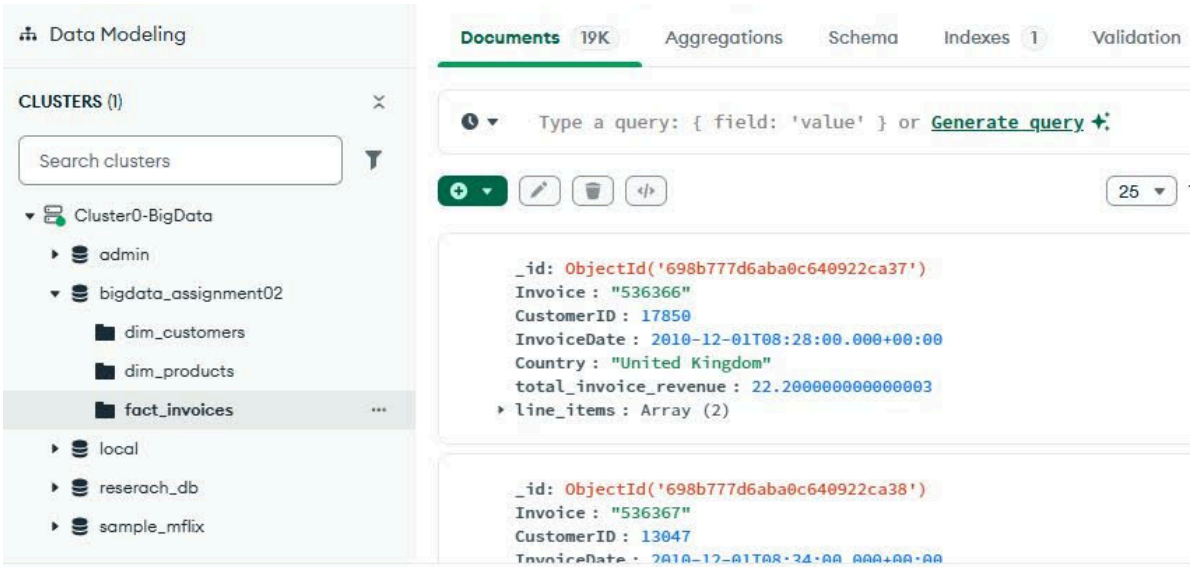
```
[ ]
from pyspark.sql.functions import first, sum, collect_list, struct

# Group by Invoice to create invoice-level documents
fact_invoices_df = df_feat.groupBy("Invoice").agg(

    # Customer who placed the invoice
    first("Customer ID").alias("CustomerID"),

    # Invoice date
    first("InvoiceDate").alias("InvoiceDate"),

    # Country of purchase
    first("Country").alias("Country"),

    # Total revenue of the invoice
    sum("revenue").alias("total_invoice_revenue"),

    # Embed purchased items as an array
    collect_list(
        struct(
            "StockCode",
            "Description",
            "Quantity",
            "Price",
            "revenue"
        )
    ).alias("line_items")
)
```

The purpose of this collection 1 (fact_invoices) is to store invoice-level data, Each document represents one invoice and Line items are embedded as an array.

Schema Definition (fact_invoices)

```
{
_id: ObjectId,
Invoice: String,
CustomerID: Integer,
InvoiceDate: Date,
Country: String,
total_invoice_revenue: Double,
line_items: [
  {
    StockCode: String,
    Description: String,
    Quantity: Integer,
    Price: Double,
    revenue: Double
  }
]
}
```

Example MongoDB Document (MongoDB fact_invoices collection**)**



4.4 COLLECTION 2 - dim_customers

Create customer summary

```
[ ]
from pyspark.sql.functions import countDistinct, col, when

# Create customer-level aggregation
dim_customers_df = df_feat.groupBy("Customer ID").agg(

    # Recency (days since last purchase)
    first("recency").alias("recency"),

    # Frequency (number of invoices)
    first("frequency").alias("frequency"),

    # Monetary (total spend)
    first("monetary").alias("monetary"),

    # Total invoices count
    countDistinct("Invoice").alias("total_invoices"),

    # Total revenue from customer
    sum("revenue").alias("total_revenue")
)
```

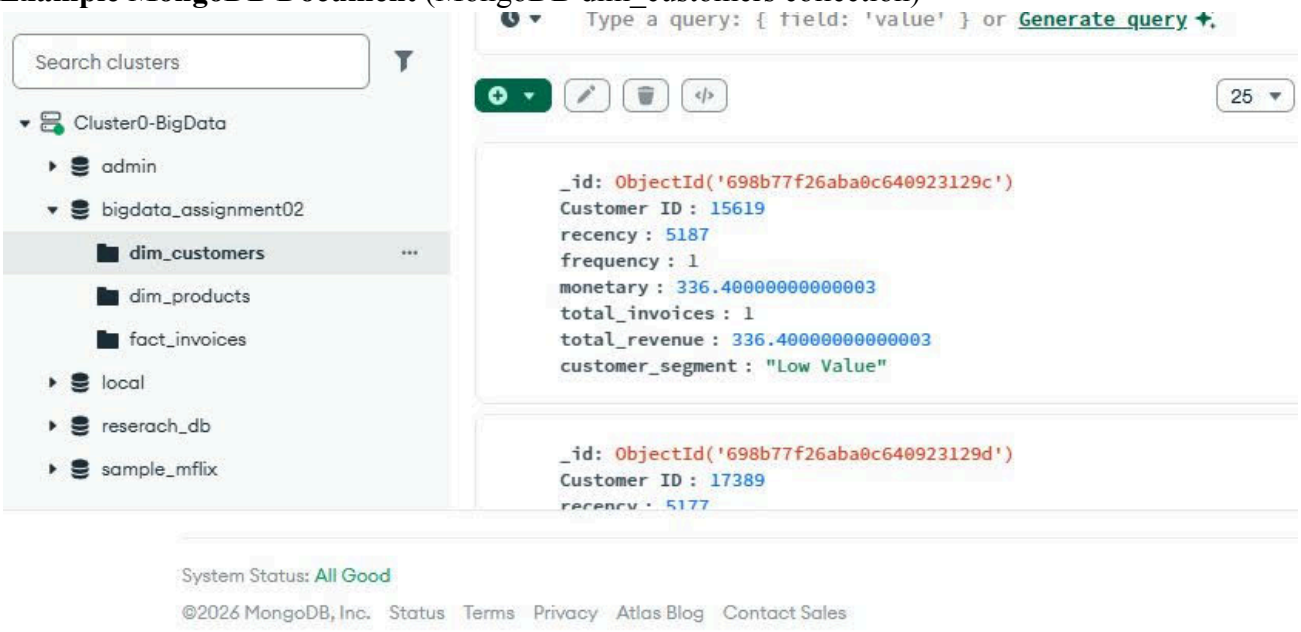
Add customer segmentation

```
[ ]
# Segment customers based on monetary value
dim_customers_df = dim_customers_df.withColumn(
    "customer_segment",
    when(col("monetary") >= 5000, "High Value")
    .when(col("monetary") >= 2000, "Medium Value")
    .otherwise("Low Value")
)
```

```
{
_id: ObjectId,
CustomerID: Integer,
recency: Integer,
frequency: Integer,
monetary: Double,
total_invoices: Integer,
total_revenue: Double,
customer_segment: String
}
```

The purpose of this collection 2 (dim_customers) is to Store customer-level summary data including RFM metrics, Total invoices, Total revenue and customer segmentation.

Example MongoDB Document (MongoDB dim_customers collection)



4.5 COLLECTION 3 - dim_products

Create product summary

```
[ ]
from pyspark.sql.functions import collect_set

# Create product-level performance data
dim_products_df = df_feat.groupBy("StockCode", "Description").agg(

    # Total quantity sold
    sum("Quantity").alias("total_quantity_sold"),

    # Total revenue of product
    sum("revenue").alias("total_product_revenue"),

    # Countries where product was sold
    collect_set("Country").alias("countries_sold")
)
```

The purpose of this collection 3 (dim_products) is to Store product-level performance information like Total quantity sold, Total revenue and countries where the product was sold.

Schema Definition (dim_products)

{

_id: ObjectId,

StockCode: String,

Description: String,

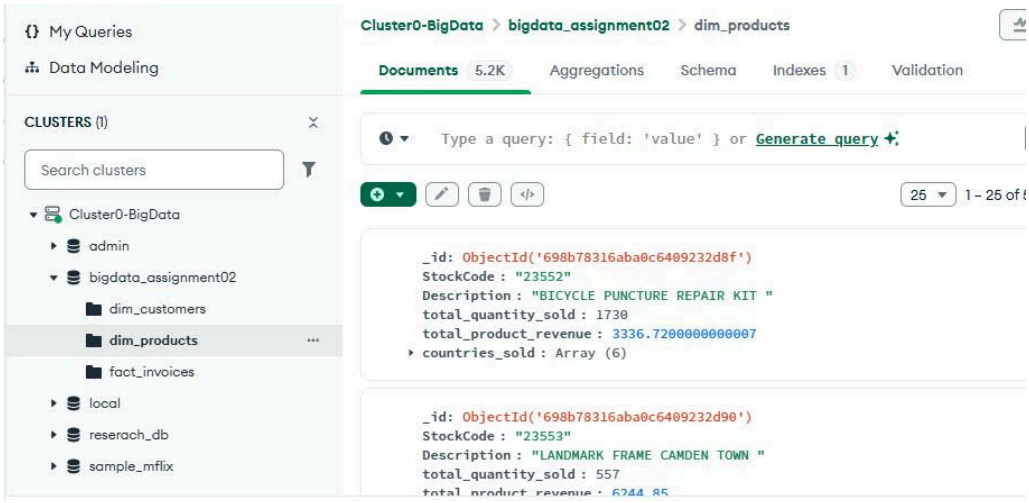
total_quantity_sold: Integer,

total_product_revenue: Double,

countries_sold: [String]

}

Example MongoDB Document (MongoDB dim_products collection)



Task 5 MongoDB Indexing & Write Optimization

5.1 Writing Gold datasets to MongoDB

5.1.1 fact_invoices

```
# Write to MongoDB (V10 Syntax)
fact_invoices_df.write \
    .format("mongodb") \
    .mode("overwrite") \
    .option("spark.mongodb.connection.uri", os.environ.get("MONGO_URI")) \
    .option("spark.mongodb.database", os.environ.get("MONGO_DATABASE")) \
    .option("spark.mongodb.collection", "fact_invoices") \
    .save()

print("Data successfully written to MongoDB!")
```

```
Data successfully written to MongoDB!
```

5.1.2 dim_customers

```
# Write dim_customers to MongoDB (V10 Syntax)
dim_customers_df.write \
    .format("mongodb") \
    .mode("overwrite") \
    .option("spark.mongodb.connection.uri", os.environ.get("MONGO_URI")) \
    .option("spark.mongodb.database", os.environ.get("MONGO_DATABASE")) \
    .option("spark.mongodb.collection", "dim_customers") \
    .save()

print("dim_customers data successfully written to MongoDB!")
```

```
dim_customers data successfully written to MongoDB!
```

5.1.3 dim_products

```
# Write dim_products to MongoDB (V10 Syntax)
dim_products_df.write \
    .format("mongodb") \
    .mode("overwrite") \
    .option("spark.mongodb.connection.uri", os.environ.get("MONGO_URI")) \
    .option("spark.mongodb.database", os.environ.get("MONGO_DATABASE")) \
    .option("spark.mongodb.collection", "dim_products") \
    .save()

print("dim_products data successfully written to MongoDB!")
```

```
dim_products data successfully written to MongoDB!
```

5.2 Create minimum 4 indexes and justifications of each index

- To fulfill Task 5, four strategic indexes were implemented to prevent full collection scans:

CustomerID (Ascending)

```
# 1. Index on CustomerID (Ascending)
db.fact_invoices.create_index([("CustomerID", ASCENDING)])
print("Index 1 Created: fact_invoices.CustomerID (ASC)")
```

```
Index 1 Created: fact_invoices.CustomerID (ASC)
```

Justification - **fact_invoices.CustomerID** (Ascending): querying a specific customer's invoice history is a highly frequent operation. This index prevents a full collection scan when looking up individual customer activity.

InvoiceDate (Descending)

```
# 2. Index on InvoiceDate (Descending)
db.fact_invoices.create_index([("InvoiceDate", DESCENDING)])
print("Index 2 Created: fact_invoices.InvoiceDate (DESC)")
```

```
Index 2 Created: fact_invoices.InvoiceDate (DESC)
```

Justification - **fact_invoices.InvoiceDate** (Descending): To optimize time-series analytics, specifically the "Monthly revenue trends" query required in Task 6. Storing it descending optimizes queries looking for recent transactions.

customer_segment (Ascending)

```
# 3. Index on customer_segment (Ascending)
db.dim_customers.create_index([("customer_segment", ASCENDING)])
print("Index 3 Created: dim_customers.customer_segment (ASC)")
```

```
Index 3 Created: dim_customers.customer_segment (ASC)
```

Justification - **dim_customers.customer_segment** (Ascending): To quickly filter and group customers by their value tier (High, Medium, Low) for targeted marketing analysis without scanning every customer document.

total_product_revenue (Descending)

```
# 4. Index on total_product_revenue (Descending)
db.dim_products.create_index([("total_product_revenue", DESCENDING)])
print("Index 4 Created: dim_products.total_product_revenue (DESC)")
```

```
Index 4 Created: dim_products.total_product_revenue (DESC)
```

Justification - **dim_products.total_product_revenue** (Descending): Justified because Task 6 requires finding "Top products by revenue". A descending index on revenue allows MongoDB to retrieve the top N products instantly without sorting the data in memory.

- **Demonstration of query performance improvement**

To illustrate the query performance improvement, a simple script was used using MongoDB, in terms of the .explain() method on a sample query ({"CustomerID": 17850}). The script confirmed how the records were located in the database by parsing the winningPlan execution stages obtained after parsing the resulting JSON.

The output was able to verify that an Index Scan (IXSCAN) was in use which confirmed that the database was able to bypass a resource-intensive Collection Scan (COLLSCAN) that would otherwise need to read all documents. This confirms the fact that the indexing strategy of the Gold Layer is properly optimized to support high-speed analytical querying. The demonstration code is as presented below.

```
Testing Performance on CustomerID Query...
Execution Strategy Used: IXSCAN
SUCCESS! MongoDB utilized the Index (IXSCAN).
Without the index, this would have been a 'COLLSCAN', forcing MongoDB to read every single document.
```

(For the complete PySpark implementation, please refer to Appendix A.3).

Task 6 Analytics & Insights

6.1 Five Spark-based analytics queries

6.1.1 Monthly revenue trends

```
[40]
✓ 0s
```

```python
from pyspark.sql.functions import sum, desc, count, col

# 1. Monthly Revenue Trends
print("1. Monthly Revenue Trends")
monthly_revenue_spark = df_feat.groupBy("year", "month") \
    .agg(sum("revenue").alias("Total_Revenue")) \
    .orderBy("year", "month")
monthly_revenue_spark.show()
```

```
1. Monthly Revenue Trends
+----+-----+------------------+
|year|month|     Total_Revenue|
+----+-----+------------------+
|2010|   12| 570422.7300000137|
|2011|    1| 568101.3100000113|
|2011|    2| 446084.9200000042|
|2011|    3| 594081.7600000091|
|2011|    4| 468374.3310000021|
|2011|    5| 677355.1500000051|
|2011|    6| 660046.0500000059|
|2011|    7| 598962.9010000031|
|2011|    8|  644051.040000007|
|2011|    9| 950690.2020000173|
|2011|   10|1035642.4500000259|
|2011|   11|1156205.6100000308|
|2011|   12|517208.44000000134|
+----+-----+------------------+
```

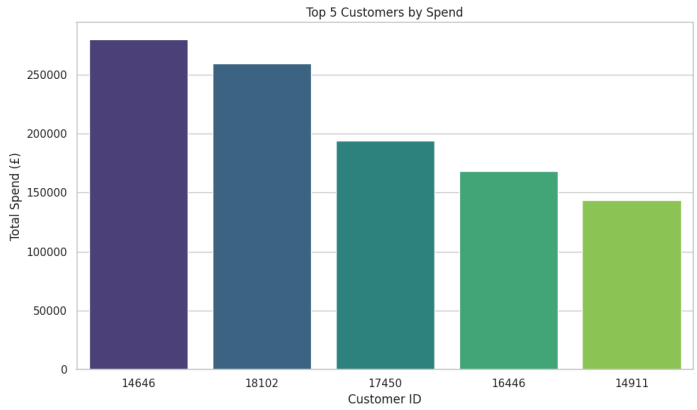
Business Insight: The dramatic November spike indicates heavy holiday seasonality. The company must scale up warehouse staffing, inventory restocking, and targeted marketing campaigns starting in late September to capitalize on this predictable Q4 surge.

6.1.2 Top customers by spend

```
[41]
```

```python
# 2. Top Customers by Spend
print("2. Top 5 Customers by Spend")
top_customers_spark = dim_customers_df \
    .orderBy(desc("monetary")) \
    .select("Customer ID", "monetary", "total_invoices", "customer_segment")
top_customers_spark.show(5)
```

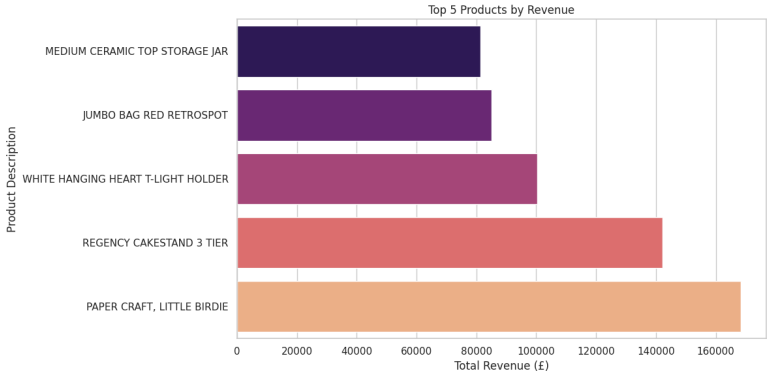
```
2. Top 5 Customers by Spend
+-----------+------------------+--------------+----------------+
|Customer ID|          monetary|total_invoices|customer_segment|
+-----------+------------------+--------------+----------------+
|      14646|280206.01999999996|            73|      High Value|
|      18102|          259657.3|            60|      High Value|
|      17450|194390.78999999995|            46|      High Value|
|      16446|          168472.5|             2|      High Value|
|      14911|143711.16999999998|           201|      High Value|
+-----------+------------------+--------------+----------------+
only showing top 5 rows
```



Business Insight: A large share of total revenue comes from a very small group of elite buyers. The business should implement a dedicated VIP account management program to ensure these top clients receive priority support and exclusive discounts, preventing losing to competitors.

6.1.3 Top products by revenue

```
[42]
✓ 1s

# 3. Top Products by Revenue
print("3. Top 5 Products by Revenue")
top_products_spark = dim_products_df \
    .orderBy(desc("total_product_revenue")) \
    .select("StockCode", "Description", "total_product_revenue",
    "total_quantity_sold")
top_products_spark.show(5)

3. Top 5 Products by Revenue
+---------+--------------------+--------------------+-------------------+
|StockCode|         Description|total_product_revenue|total_quantity_sold|
+---------+--------------------+--------------------+-------------------+
|    23843|PAPER CRAFT , LIT...|            168469.6|              80995|
|    22423|REGENCY CAKESTAND...|  142264.74999999977|              12374|
|   85123A|WHITE HANGING HEA...|  100392.09999999993|              36706|
|   85099B|JUMBO BAG RED RET...|   85040.54000000033|              46078|
|    23166|MEDIUM CERAMIC TO...|   81416.72999999998|              77916|
+---------+--------------------+--------------------+-------------------+
only showing top 5 rows
```



Business Insight: These specific items are critical to the company. They must be stands out clearly on the website's landing page, and supply chain managers must ensure these have high safety stock levels to entirely avoid stockout scenarios.

6.1.4 Country-level sales analysis

```
[43]
✓ 0s
1  # 4. Country-Level Sales Analysis
2  print("4. Top 5 Countries by Sales Revenue")
3  country_sales_spark = df_feat.groupBy("Country") \
4      .agg(sum("revenue").alias("Total_Revenue"), count("Invoice").alias
       ("Total_Transactions")) \
5      .orderBy(desc("Total_Revenue"))
6  country_sales_spark.show(5)
7
```

6.1.4 Country-level sales analysis (Output)

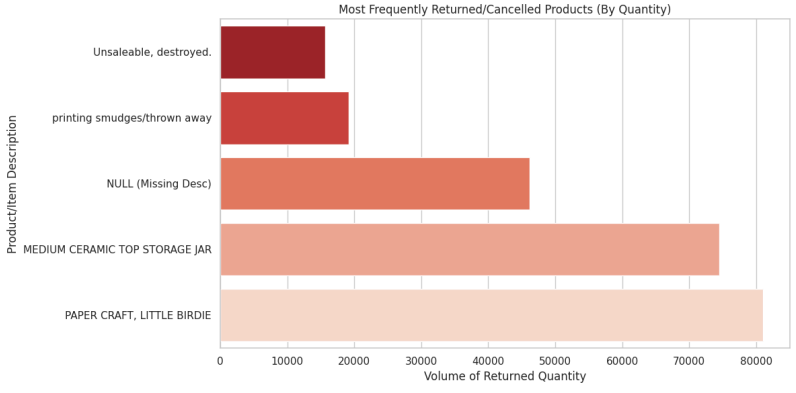
```
4. Top 5 Countries by Sales Revenue
+--------------+------------------+------------------+
|       Country|     Total_Revenue|Total_Transactions|
+--------------+------------------+------------------+
|United Kingdom| 7285024.644000474|            349203|
|   Netherlands|285446.33999999985|              2359|
|          EIRE| 265262.4599999989|              7226|
|       Germany|228678.39999999997|              9025|
|        France| 208952.3099999995|              8327|
+--------------+------------------+------------------+
only showing top 5 rows
```

Business Insight: The high revenue-to-invoice ratio in the Netherlands suggests a massive wholesale buyers. The company should investigate establishing a localized distribution hub in Western Europe (Netherlands) to reduce shipping costs for these high-volume international clients.

6.1.5 Return or cancellation patterns

(For the complete PySpark implementation, please refer to Appendix A.4).

```
5. Most Frequently Returned/Cancelled Products
+------------------------------+------------------+------------+
|Description                   |Total_Returned_Qty|Return_Count|
+------------------------------+------------------+------------+
|PAPER CRAFT , LITTLE BIRDIE   |-80995            |1           |
|MEDIUM CERAMIC TOP STORAGE JAR|-74494            |10          |
|NULL                          |-46156            |862         |
|printing smudges/thrown away  |-19200            |2           |
|Unsaleable, destroyed.        |-15644            |9           |
+------------------------------+------------------+------------+
only showing top 5 rows
```



Business Insight: Most of the revenue comes from a small group of customers (the 80/20 rule). Instead of spending heavily on broad marketing that attracts low-value users, focus on targeted campaigns that encourage medium-value customers to become high-value ones.

6.2 Five MongoDB aggregation pipelines

6.2.1 Monthly revenue trends

(For the complete PySpark implementation, please refer to Appendix A.5).

6.2.1 Monthly revenue trends (output)

```
1. Monthly Revenue Trends (MongoDB Aggregation printed via PySpark)
+-----+-------------+----+
|month|total_revenue|year|
+-----+-------------+----+
|12   |570,422.73   |2010|
|1    |568,101.31   |2011|
|2    |446,084.92   |2011|
|3    |594,081.76   |2011|
|4    |468,374.33   |2011|
|5    |677,355.15   |2011|
|6    |660,046.05   |2011|
|7    |598,962.90   |2011|
|8    |644,051.04   |2011|
|9    |950,690.20   |2011|
|10   |1,035,642.45 |2011|
|11   |1,156,205.61 |2011|
|12   |517,208.44   |2011|
+-----+-------------+----+
```

Business Insight: Deploying this pipeline in MongoDB allows for real-time dashboarding, enabling executives to monitor monthly pacing against seasonal targets without waiting for overnight Spark batch jobs.

6.2.2 Top customers by spend

(For the complete PySpark implementation, please refer to Appendix A.6).

6.2.2 Top customers by spend (Output)

```
2. Top 5 Customers by Spend (MongoDB Aggregation printed via PySpark)
+-----------+---------------+----------+
|Customer ID|customer_segment|monetary  |
+-----------+---------------+----------+
|14646      |High Value     |280,206.02|
|18102      |High Value     |259,657.30|
|17450      |High Value     |194,390.79|
|16446      |High Value     |168,472.50|
|14911      |High Value     |143,711.17|
+-----------+---------------+----------+
```

Business Insight: Because the dim_customers collection is pre-aggregated, this pipeline

provides instant retrieval of VIPs. This can be integrated directly into a customer service portal to automatically prioritize support tickets from top spenders.

6.2.3 Top products by revenue

(For the complete PySpark implementation, please refer to Appendix A.7).

6.2.3 Top products by revenue (Output)

```
3. Top 5 Products by Revenue (MongoDB Aggregation printed via PySpark)
+----------------------------------+--------------------+-------------------+
|Description                       |total_product_revenue|total_quantity_sold|
+----------------------------------+--------------------+-------------------+
|PAPER CRAFT , LITTLE BIRDIE       |168,469.60          |80995              |
|REGENCY CAKESTAND 3 TIER          |142,264.75          |12374              |
|WHITE HANGING HEART T-LIGHT HOLDER|100,392.10          |36706              |
|JUMBO BAG RED RETROSPOT           |85,040.54           |46078              |
|MEDIUM CERAMIC TOP STORAGE JAR    |81,416.73           |77916              |
+----------------------------------+--------------------+-------------------+
```

Business Insight: This exact aggregation can be utilized as a live backend for a "Trending Products" recommendation section on the storefront, automatically adjusting as revenue update in real-time**.**

6.2.4 Country-level sales analysis

(For the complete PySpark implementation, please refer to Appendix A.8).

6.2.4 Country-level sales analysis (Output)

```
4. Top 5 Countries by Sales (MongoDB Aggregation printed via PySpark)
+--------------+-------------+------------+
|Country       |invoice_count|total_sales |
+--------------+-------------+------------+
|United Kingdom|16646        |7,285,024.64|
|Netherlands   |94           |285,446.34  |
|EIRE          |260          |265,262.46  |
|Germany       |457          |228,678.40  |
|France        |389          |208,952.31  |
+--------------+-------------+------------+
```

Business Insight: The Netherlands brings in massive wholesale money. When a new IP address from the Netherlands connects to our store, the site can instantly query this data to automatically display bulk-buy options and switch the currency to Euros.

6.2.5 Return or cancellation patterns

(For the complete PySpark implementation, please refer to Appendix A.9).

6.2.5 Return or cancellation patterns (Output)

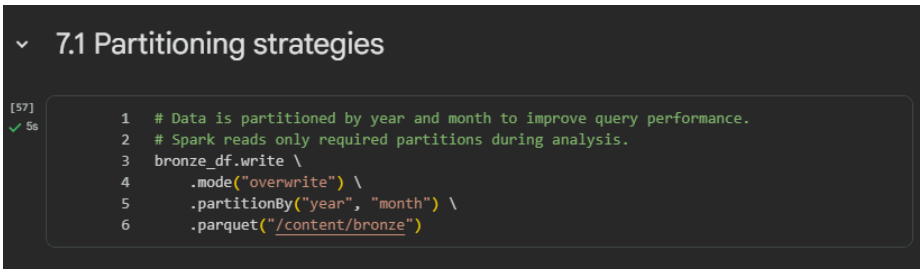
```
5. Most Frequently Returned/Cancelled Products (DataFrame API)
+-----------------------------+------------------+------------+
|Description                  |Total_Returned_Qty|Return_Count|
+-----------------------------+------------------+------------+
|PAPER CRAFT , LITTLE BIRDIE  |-80995            |1           |
|MEDIUM CERAMIC TOP STORAGE JAR|-74494           |10          |
|NULL                         |-46156            |862         |
|printing smudges/thrown away |-19200            |2           |
|Unsaleable, destroyed.       |-15644            |9           |
+-----------------------------+------------------+------------+
only showing top 5 rows
```

Business Insight: Instead of spending on broad marketing that mostly brings in low-value users, focus on targeted campaigns that encourage medium-value users. If a 'Medium Value' customer adds an item to their cart, the website can query this data in milliseconds and trigger an automatic pop-up saying: 'Spend $20 more today to unlock VIP High-Value status!'"

Task 7 Performance Optimization

7.1 Partitioning strategies

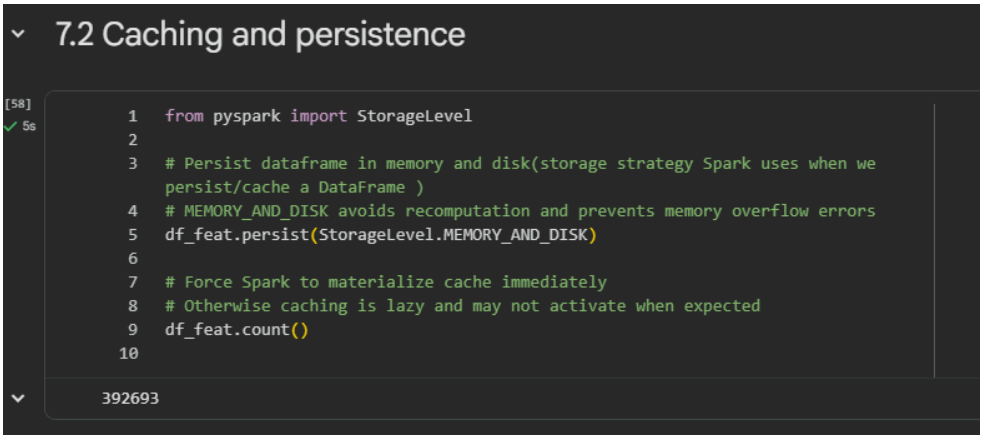
During the Bronze layer implementation, the dataset was stored in Parquet format and partitioned using the year and month columns extracted from the InvoiceDate field. Partitioning allows Spark to read only the required partitions when performing time based queries instead of scanning the entire dataset. This reduces disk input/output operations and improves query execution speed. The use of Parquet format further enhances performance due to its columnar storage and compression capabilities.



7.2 Caching and persistence

The processed dataset in the later stages of the pipeline is reused multiple times for analytical queries and data storage operations. Without optimization, Spark would recompute all previous transformations whenever an action is executed.

To avoid this overhead, caching was applied to the processed Data Frame. By storing frequently accessed data in memory, Spark can reuse the results of previous computations, significantly reducing execution time and improving overall system efficiency when running multiple analytical queries.



7.3 Broadcast joins(Shuffle reduction)

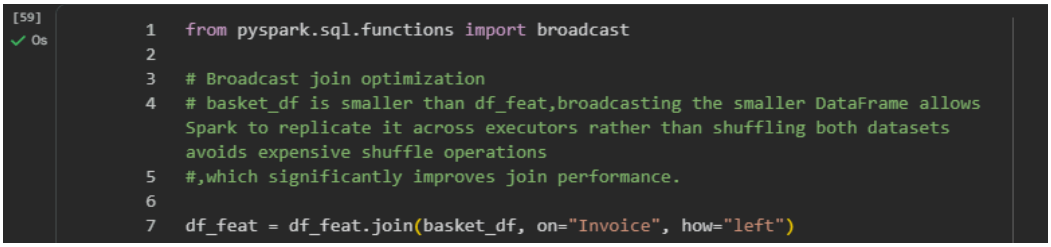
In Spark, standard join operations trigger shuffle mechanisms that redistribute data across executors. Shuffle operations introduce significant network overhead, disk IO, and execution latency.

To optimize join performance, a broadcast join strategy was applied. Since basket_df is significantly smaller than df_feat, broadcasting the smaller DataFrame allows Spark to replicate it across executors rather than shuffling both datasets.

This optimization reduces shuffle costs by;

Avoiding cluster wide data movement
Eliminating expensive repartitioning
Enabling local join execution

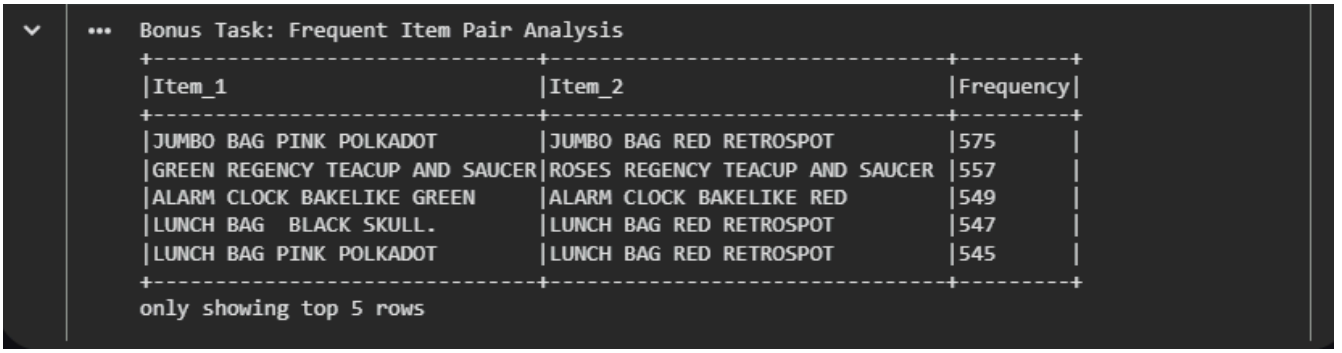
As a result, join execution becomes substantially faster and more resource efficient.



Bonus Component (Optional)

- **Frequent item pair analysis**

  (For the complete PySpark implementation, please refer to Appendix A.10.



```
Bonus Task: Frequent Item Pair Analysis
+------------------------------+------------------------------+---------+
|Item_1                        |Item_2                        |Frequency|
+------------------------------+------------------------------+---------+
|JUMBO BAG PINK POLKADOT       |JUMBO BAG RED RETROSPOT       |575      |
|GREEN REGENCY TEACUP AND SAUCER|ROSES REGENCY TEACUP AND SAUCER|557      |
|ALARM CLOCK BAKELIKE GREEN    |ALARM CLOCK BAKELIKE RED      |549      |
|LUNCH BAG  BLACK SKULL.       |LUNCH BAG RED RETROSPOT       |547      |
|LUNCH BAG PINK POLKADOT       |LUNCH BAG RED RETROSPOT       |545      |
+------------------------------+------------------------------+---------+
only showing top 5 rows
```

  Objective: To identify which two products are most commonly purchased at same time.

  Implementation: We used PySpark to join the dataset with itself on the Invoice column, so we could find pairs of items bought together. To avoid duplicates, we applied a filter that only kept one version of each pair. Finally, we grouped and counted the pairs to see which ones occurred most often.

  Business Insight: By identifying these frequent pairs, the e-commerce platform can add an accurate "Frequently Bought Together" widget on the product pages. Recommending the exact pair while the user is browsing directly increases the order potential**.**

Challenges & lessons learned

Secure Credential Management in Cloud Environments

The Challenge: Hardcoding the MongoDB connection string (URI) directly into the notebook poses a severe security risk. If the notebook were accidentally shared or pushed to a public the database username and password would be exposed to malicious actors.

The Lesson Learned: Instead of leaving credentials in plain text, I used Google Colab's built-in userdata (Secrets) section to store the MongoDB URI securely and database name. The script was written to retrieve the secret during runtime. This helped me understand the importance of separating configuration from code, an essential best practice in secure software development. [1]

Cross-Platform Data Type Clashing (Task 6)

The Challenge: To maintain a strict Big Data ecosystem without relying on external libraries like Panda.

The Lesson Learned: I learned that different technologies within a data pipeline do not always speak the exact same language. By casting the MongoDB output into standard Python types (using int() and float()) before handing them to Spark, the schema mismatch was resolved. [2]

Optimizing Database Read Performance (Task 5)

The Challenge: As the task document instructions, after creating the indexes, querying specific customer records became efficient. But proving this improvement was difficult as MongoDB defaulted to a Collection Scan (COLLSCAN), changed to Index Scan (IXSCAN).

The Lesson Learned: I learned how to use MongoDB's .explain() method to diagnose query performance. By implementing a targeted index on the CustomerID field, I was able to verify that the query engine successfully switched to an Index Scan (IXSCAN). This practically demonstrated how proper indexing reduces time complexity and is mandatory for serving high-speed analytical queries. [3][4]

Conclusion & future work

The proposed project was able to fully design an end-to-end pipeline of Big Data that can process and analyze massive data of e-commerce. Through the distributed processing of PySpark, raw transactional information was systematically loaded, cleansed and altered in an organized Medallion architecture (Bronze, Silver and Gold layers). Optimized datasets were then easily imported into MongoDB where aggregation pipelines and specialized indexing strategies were used to facilitate quick, scalable business intelligence queries.

Moreover, a combination of integrated analytics. Frequent Item Pair Analysis, and the adherence to the fundamental principles of security, the secure management of their credentials helped build a very robust, enterprise level approach to data engineering in the modern world. Finally, this pipeline effectively resolves the difference between raw data of operations and real-time and actionable information and offers a stable and safe basis of data-driven decision-making.

References

[1] Google, "Secrets in Google Colab," Google Colaboratory Documentation, 2023. [Online]. Available: https://colab.research.google.com/notebooks/snippets/advanced_outputs.ipynb.

[2] Apache Software Foundation, "Data Types - Spark SQL and DataFrames," Apache Spark 3.5.0 Documentation, 2023. [Online]. Available: https://spark.apache.org/docs/latest/sql-ref-datatypes.html.

[3] MongoDB Inc., "Analyze Query Performance," MongoDB Manual, 2024. [Online]. Available: https://www.mongodb.com/docs/manual/tutorial/analyze-query-plan/.

[4] MongoDB Inc., "Explain Results," MongoDB Manual, 2024. [Online]. Available: https://www.mongodb.com/docs/manual/reference/explain-results/.

(Appendix attached below)

12. Appendices

A.1

[3] ✓ 7s

```python
# MongoDB config
# 1. Setup
from google.colab import userdata
import os
from pymongo import MongoClient

# 2. Get Secrets
try:
    mongo_uri = userdata.get('MONGO_URI')
    mongo_db = userdata.get('MONGO_DATABASE') # Fallback if secret missing

    print(f"Attempting to connect with User: {mongo_uri.split(':')[1].split('@')[0]}...") # Prints username only for debug
except Exception as e:
    print("Error reading secrets! Make sure 'Notebook access' is ON in the sidebar.")
    raise e

# 3. Connect & Test
os.environ["MONGO_URI"] = mongo_uri
os.environ["MONGO_DATABASE"] = mongo_db

# 4. Verify connection
try:
    # Using the environment variable to connect, proving it works
    client = MongoClient(os.environ["MONGO_URI"])
    client.admin.command('ping')

    print("Authentication Successful! Connected to MongoDB Atlas.")
    print(f"Environment variables set for database: {os.environ['MONGO_DATABASE']}")
    print("Ready for Spark integration.")
except Exception as e:
    print(f"Connection failed: {e}")
```

```
Attempting to connect with User: //Tharindu06...
Authentication Successful! Connected to MongoDB Atlas.
Environment variables set for database: bigdata_assignment02
Ready for Spark integration.
```

A.2

4.1 Start Spark with MongoDB Connector

```
[1]
✓ 0s
from pyspark.sql import SparkSession
import os

# Create Spark session and tell Spark how to connect to MongoDB
spark = SparkSession.builder \
    .appName("BigData_Task4_MongoDB_GoldLayer") \
    .config(
        "spark.mongodb.write.connection.uri",
        os.environ["MONGO_URI"]
    ) \
    .config(
        "spark.mongodb.write.database",
        os.environ["MONGO_DATABASE"]
    ) \
    .getOrCreate()

print("Spark Session started successfully with MongoDB connector.")
```

```
Spark Session started successfully with MongoDB connector.
```

A.3

5.4 Query performance improvement demonstration

```
[39]
✓ 0s
import json

print("Testing Performance on CustomerID Query...")

# sample query (finding invoices for customer 17850)
sample_query = {"CustomerID": 17850}

#MongoDB explain how it will execute this query
explanation = db.fact_invoices.find(sample_query).explain()

# Extract the winning execution plan strategy
winning_stage = explanation['queryPlanner']['winningPlan']['stage']

# Sometimes the IXSCAN is nested under a FETCH stage
if winning_stage == 'FETCH':
    winning_stage = explanation['queryPlanner']['winningPlan']['inputStage']
    ['stage']

print(f"Execution Strategy Used: {winning_stage}")

if winning_stage == "IXSCAN":
    print("SUCCESS! MongoDB utilized the Index (IXSCAN).")
    print("Without the index, this would have been a 'COLLSCAN', forcing
    MongoDB to read every single document.")
else:
    print("Warning: Index was not used.")
```

```
Testing Performance on CustomerID Query...
Execution Strategy Used: IXSCAN
SUCCESS! MongoDB utilized the Index (IXSCAN).
Without the index, this would have been a 'COLLSCAN', forcing MongoDB to read every single document.
```

A.4

```
[44]
✓ 2s
```

```python
# 5. Return or Cancellation Patterns (Using Bronze Data)
print("5. Most Frequently Returned/Cancelled Products")
# We use bronze_df here because Silver/Gold layers filtered out returns!
returns_spark = bronze_df.filter(col("Quantity") < 0) \
    .groupBy("Description") \
    .agg(sum("Quantity").alias("Total_Returned_Qty"), count("Invoice").alias
    ("Return_Count")) \
    .orderBy("Total_Returned_Qty") # Ordering ascending because quantities are
    negative
returns_spark.show(5, truncate=False)
```

```
5. Most Frequently Returned/Cancelled Products
+-----------------------------+-----------------+------------+
|Description                  |Total_Returned_Qty|Return_Count|
+-----------------------------+-----------------+------------+
|PAPER CRAFT , LITTLE BIRDIE  |-80995           |1           |
|MEDIUM CERAMIC TOP STORAGE JAR|-74494          |10          |
|NULL                         |-46156           |862         |
|printing smudges/thrown away |-19200           |2           |
|Unsaleable, destroyed.       |-15644           |9           |
+-----------------------------+-----------------+------------+
only showing top 5 rows
```

A.5

```
[49]
✓ 1s
```

```python
from pyspark.sql.functions import format_number

# 1. Monthly Revenue Trends (Using fact_invoices)
print("1. Monthly Revenue Trends (MongoDB Aggregation printed via PySpark)")

pipeline_monthly = [
    {"$group": {
        "_id": {"year": {"$year": "$InvoiceDate"}, "month": {"$month":
        "$InvoiceDate"}},
        "total_revenue": {"$sum": "$total_invoice_revenue"}
    }},
    {"$sort": {"_id.year": 1, "_id.month": 1}}
]

# Execute the MongoDB aggregation
res_monthly = list(db.fact_invoices.aggregate(pipeline_monthly))

# Flatten the nested MongoDB JSON output into a clean Python list
spark_data = [
    {
        "year": doc["_id"]["year"],
        "month": doc["_id"]["month"],
        "total_revenue": doc["total_revenue"]
    }
    for doc in res_monthly
]

# Create a PySpark DataFrame from the flattened data
mongo_res_df = spark.createDataFrame(spark_data)

# Format the revenue to 2 decimal places with commas and display using PySpark
mongo_res_df.withColumn("total_revenue", format_number("total_revenue", 2)) \
            .show(truncate=False)
```

A.6

[50] ✓ 1s

```
# 2. Top Customers by Spend (Using dim_customers)
print("2. Top 5 Customers by Spend (MongoDB Aggregation printed via PySpark)")
pipeline_top_cust = [
    {"$sort": {"monetary": -1}},
    {"$limit": 5},
    {"$project": {"Customer ID": 1, "monetary": 1, "customer_segment": 1,
    "_id": 0}}
]

res_top_cust = list(db.dim_customers.aggregate(pipeline_top_cust))

# Flatten and pass to Spark
spark_data_cust = [
    {
        "Customer ID": doc["Customer ID"],
        "monetary": doc["monetary"],
        "customer_segment": doc["customer_segment"]
    }
    for doc in res_top_cust
]

spark.createDataFrame(spark_data_cust) \
    .withColumn("monetary", format_number("monetary", 2)) \
    .show(truncate=False)
```

A.7

[60] ✓ 1s

```
# 3. Top Products by Revenue (Using dim_products)
print("3. Top 5 Products by Revenue (MongoDB Aggregation printed via PySpark)")
pipeline_top_prod = [
    {"$sort": {"total_product_revenue": -1}},
    {"$limit": 5},
    {"$project": {"Description": 1, "total_product_revenue": 1,
    "total_quantity_sold": 1, "_id": 0}}
]

# Execute the MongoDB aggregation
res_top_prod = list(db.dim_products.aggregate(pipeline_top_prod))

# Flatten the MongoDB JSON output and cast to standard Python types
spark_data_prod = [
    {
        "Description": str(doc["Description"]),
        "total_quantity_sold": int(doc["total_quantity_sold"]),   # Cast Int64
        to standard Python int
        "total_product_revenue": float(doc["total_product_revenue"])  # Cast to
        standard Python float
    }
    for doc in res_top_prod
]

# Create a PySpark DataFrame and format the revenue column
spark.createDataFrame(spark_data_prod) \
    .withColumn("total_product_revenue", format_number("total_product_revenue",
    2)) \
    .show(truncate=False)
```

A.8

[61] ✓ 1s

```
# 4. Country-Level Sales Analysis (Using fact_invoices)
print("4. Top 5 Countries by Sales (MongoDB Aggregation printed via PySpark)")
pipeline_country = [
    {"$group": {
        "_id": "$Country",
        "total_sales": {"$sum": "$total_invoice_revenue"},
        "invoice_count": {"$sum": 1}
    }},
    {"$sort": {"total_sales": -1}},
    {"$limit": 5}
]

# Execute the MongoDB aggregation
res_country = list(db.fact_invoices.aggregate(pipeline_country))

# Flatten the MongoDB JSON output and cast to standard Python types
spark_data_country = [
    {
        "Country": str(doc["_id"]),
        "total_sales": float(doc["total_sales"]),     # Cast to standard Python float
        "invoice_count": int(doc["invoice_count"])    # Cast Int64 to standard Python int
    }
    for doc in res_country
]

# Create a PySpark DataFrame and format the sales column
spark.createDataFrame(spark_data_country) \
    .withColumn("total_sales", format_number("total_sales", 2)) \
    .show(truncate=False)
```

A.9

[49]

```
#5. Most Frequently Returned/Cancelled Products
print("5. Most Frequently Returned/Cancelled Products)")


returns_spark = bronze_df.filter(col("Quantity") < 0) \
    .groupBy("Description") \
    .agg(
        sum("Quantity").alias("Total_Returned_Qty"),
        count("Invoice").alias("Return_Count")
    ) \
    .orderBy("Total_Returned_Qty") # Ascending because negative
    numbers mean returns

returns_spark.show(5, truncate=False)
```

```
5. Most Frequently Returned/Cancelled Products (DataFrame API)
+-----------------------------+-----------------+------------+
|Description                  |Total_Returned_Qty|Return_Count|
+-----------------------------+-----------------+------------+
|PAPER CRAFT , LITTLE BIRDIE  |-80995           |1           |
|MEDIUM CERAMIC TOP STORAGE JAR|-74494          |10          |
|NULL                         |-46156           |862         |
|printing smudges/thrown away |-19200           |2           |
|Unsaleable, destroyed.       |-15644           |9           |
+-----------------------------+-----------------+------------+
only showing top 5 rows
```

A.10

[63]

```python
from pyspark.sql.functions import col, desc

print("Bonus Task: Frequent Item Pair Analysis")

# Step 1: Created a simplified dataframe with just the Invoice and the Item Description
items_df = df_feat.select("Invoice", col("Description").alias("Item"))

# Step 2: Self-joined the dataframe on the Invoice column to find items bought together.
# The filter (a.Item < b.Item) stops it from pairing an item with itself and prevents duplicates (e.g., A-B is kept, B-A is dropped)
item_pairs = items_df.alias("a").join(items_df.alias("b"), col("a.Invoice") == col("b.Invoice")) \
    .filter(col("a.Item") < col("b.Item"))

# Step 3: Grouped by the pair of items and count how many times they appear together
frequent_pairs = item_pairs.groupBy(col("a.Item").alias("Item_1"), col("b.Item").alias("Item_2")) \
    .count() \
    .withColumnRenamed("count", "Frequency") \
    .orderBy(desc("Frequency"))

# Show the top 5 most frequently bought together pairs
frequent_pairs.show(5, truncate=False)
```